```
!pip -q install datasets pandas numpy scikit-learn matplotlib seaborn tensorflow
```

```
from datasets import load_dataset

dataset = load_dataset(
    "lacg030175/CICIDS2017",
    "temporal"
)

print(dataset)
print("Available splits:", list(dataset.keys()))
```

```
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate lim
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set
README.md: 100% 3.82k/3.82k [00:00<00:00, 218kB/s]
temporal/train-00000-of- 265MB / 265MB, 24.5MB/s
00001.parquet: reconstructing file: 100%
temporal/train-00000-of-00001.parquet: downloading bytes: 250MB, 22.3MB/s
temporal/test-00000-of- 72.3MB / 72.3MB, 6.90MB/s
00001.parquet: reconstructing file: 100%
temporal/test-00000-of-00001.parquet: downloading bytes: 68.3MB, 6.43MB/s
Generating train split: 100% 2125158/2125158 [00:16<00:00, 126950.51 examples/s]
Generating test split: 100% 702718/702718 [00:05<00:00, 127915.01 examples/s]
DatasetDict({
    train: Dataset({
        features: ['Destination Port', 'Flow Duration', 'Total Fwd Packets', 'Total Backward Packets', 'Total Le
        num_rows: 2125158
    })
    test: Dataset({
        features: ['Destination Port', 'Flow Duration', 'Total Fwd Packets', 'Total Backward Packets', 'Total Le
        num_rows: 702718
    })
})
```

```
for split_name, split_data in dataset.items():
    print(f"\n{split_name}: {split_data.num_rows:,} rows")
    print("Columns:", split_data.column_names)

    print("\nSample records:")
    print(split_data.select(range(min(5, split_data.num_rows))).to_pandas())
```

```
   Fwd Packet Length Std  ...  Active Mean  Active Std  Active Max  \
0                    0.0  ...          0.0         0.0           0
1                    0.0  ...          0.0         0.0           0
```

```
0                        6448                         1152
1                        6448                         5056
2                           0                            0
3                         616                            0
4                           0                            0

   Fwd Packet Length Max  Fwd Packet Length Min  Fwd Packet Length Mean  \
0                    403                      0                   201.5
1                    403                      0                   201.5
2                      0                      0                     0.0
3                     28                     28                    28.0
4                      0                      0                     0.0

   Fwd Packet Length Std  ...   Active Mean    Active Std  Active Max  \
0             204.724205  ...  3.594286e+02  1.199802e+01         380
1             204.724205  ...  3.202857e+02  1.574499e+01         330
2               0.000000  ...  9.361829e+06  7.324646e+06    18900000
3               0.000000  ...  0.000000e+00  0.000000e+00           0
4               0.000000  ...  0.000000e+00  0.000000e+00           0

   Active Min   Idle Mean      Idle Std  Idle Max  Idle Min   Label  label
0         343  16100000.0  4.988048e+05  16400000  15400000  BENIGN      0
1         285  16100000.0  4.987937e+05  16400000  15400000  BENIGN      0
2          19  12200000.0  6.935824e+06  20800000   5504997  BENIGN      0
3           0         0.0  0.000000e+00         0         0  BENIGN      0
4           0         0.0  0.000000e+00         0         0  BENIGN      0

[5 rows x 80 columns]
```

```
print("Training label values:")
print(dataset["train"].unique("label"))

print("\nTest label values:")
print(dataset["test"].unique("label"))

print("\nOriginal multiclass labels:")
print(dataset["train"].unique("Label"))
```

```
Training label values:
[0, 1]

Test label values:
[0, 1]

Original multiclass labels:
['BENIGN', 'FTP-Patator', 'SSH-Patator', 'DoS slowloris', 'DoS Slowhttptest', 'DoS Hulk', 'DoS GoldenEye', 'Hear
```

```
import numpy as np
import pandas as pd

# Select rows across the full length of each existing split
def ordered_sample(split, sample_size):
    n = len(split)
    indices = np.linspace(
        0, n - 1, min(n, sample_size), dtype=np.int64
    )
    return split.select(indices.tolist()).to_pandas()

train_df = ordered_sample(dataset["train"], 150_000)
test_df = ordered_sample(dataset["test"], 50_000)

print("Training sample:", train_df.shape)
print("Testing sample:", test_df.shape)

print("\nBinary label distribution in training sample:")
print(train_df["label"].value_counts(dropna=False))

print("\nBinary label distribution in test sample:")
print(test_df["label"].value_counts(dropna=False))

print("\nOriginal attack-type labels:")
print(train_df["Label"].value_counts().head(15))
```

```
Training sample: (150000, 80)
Testing sample: (50000, 80)

Binary label distribution in training sample:
```

```
label
0    131038
1     18962
Name: count, dtype: int64

Binary label distribution in test sample:
label
0    29481
1    20519
Name: count, dtype: int64

Original attack-type labels:
Label
BENIGN                        131038
DoS Hulk                       16256
DoS GoldenEye                    743
FTP-Patator                      576
SSH-Patator                      425
DoS slowloris                    395
DoS Slowhttptest                 388
Web Attack ? Brute Force         116
Web Attack ? XSS                  55
Infiltration                       4
Web Attack ? Sql Injection         3
Heartbleed                         1
Name: count, dtype: int64
```

```
from sklearn.preprocessing import StandardScaler

# Convert binary labels to 0/1
def encode_binary_labels(series):
    if pd.api.types.is_numeric_dtype(series):
        values = set(series.dropna().unique())
        if not values.issubset({0, 1}):
            raise ValueError(f"Unexpected numeric labels: {values}")
        return series.astype(int).to_numpy()

    text = series.astype(str).str.strip().str.lower()
    unknown = ~text.isin(["benign", "attack", "0", "1"])

    if unknown.any():
        raise ValueError(
            "Unexpected label values: "
            + str(text[unknown].unique()[:10])
        )

    return text.map({
        "benign": 0, "attack": 1, "0": 0, "1": 1
    }).astype(int).to_numpy()


y_all = encode_binary_labels(train_df["label"])
y_test = encode_binary_labels(test_df["label"])

# Use numeric network-flow features only
excluded = ["Label", "label"]
feature_df = train_df.drop(columns=excluded, errors="ignore")
test_features = test_df.drop(columns=excluded, errors="ignore")

feature_df = feature_df.apply(pd.to_numeric, errors="coerce")
test_features = test_features.apply(pd.to_numeric, errors="coerce")

# Keep only features that contain data in the training sample
feature_cols = feature_df.columns[
    feature_df.notna().any()
].tolist()

feature_df = feature_df[feature_cols]
test_features = test_features.reindex(columns=feature_cols)

# Replace infinite values with missing values
feature_df = feature_df.replace([np.inf, -np.inf], np.nan)
test_features = test_features.replace([np.inf, -np.inf], np.nan)

# Chronological split within the training portion: first 80% / last 20%
cut = int(len(feature_df) * 0.8)
```

```
X_train_raw = feature_df.iloc[:cut].copy()
X_val_raw = feature_df.iloc[cut:].copy()

y_train = y_all[:cut]
y_val = y_all[cut:]

# Fit imputation values using training data only
medians = X_train_raw.median().fillna(0)

X_train_raw = X_train_raw.fillna(medians)
X_val_raw = X_val_raw.fillna(medians)
X_test_raw = test_features.fillna(medians)

# Fit scaler using training data only
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train_raw).astype(np.float32)
X_val = scaler.transform(X_val_raw).astype(np.float32)
X_test = scaler.transform(X_test_raw).astype(np.float32)

print("Features:", len(feature_cols))
print("Training:", X_train.shape)
print("Validation:", X_val.shape)
print("Testing:", X_test.shape)

print("\nTraining labels:", np.unique(y_train, return_counts=True))
print("Validation labels:", np.unique(y_val, return_counts=True))
print("Test labels:", np.unique(y_test, return_counts=True))
```

```
Features: 78
Training: (120000, 78)
Validation: (30000, 78)
Testing: (50000, 78)

Training labels: (array([0, 1]), array([101173,  18827]))
Validation labels: (array([0, 1]), array([29865,   135]))
Test labels: (array([0, 1]), array([29481, 20519]))
```

```
from sklearn.linear_model import SGDClassifier
from sklearn.metrics import (
    accuracy_score,
    f1_score,
    classification_report,
    confusion_matrix
)

baseline = SGDClassifier(
    loss="log_loss",
    max_iter=10,
    tol=None,
    class_weight="balanced",
    random_state=42
)

baseline.fit(X_train, y_train)

baseline_val_pred = baseline.predict(X_val)

print("Baseline validation accuracy:",
      round(accuracy_score(y_val, baseline_val_pred), 4))

print("Baseline validation macro-F1:",
      round(f1_score(y_val, baseline_val_pred,
                     average="macro", zero_division=0), 4))

print("\nBaseline validation report:")
print(classification_report(
    y_val,
    baseline_val_pred,
    labels=[0, 1],
    target_names=["Benign", "Attack"],
    zero_division=0
))
```

```
Baseline validation accuracy: 0.858
Baseline validation macro-F1: 0.463
```

```
Baseline validation report:
              precision    recall  f1-score   support

      Benign       0.99      0.86      0.92     29865
      Attack       0.00      0.04      0.00       135

    accuracy                           0.86     30000
   macro avg       0.50      0.45      0.46     30000
weighted avg       0.99      0.86      0.92     30000
```

```
import tensorflow as tf

from tensorflow.keras import Sequential
from tensorflow.keras.layers import (
    Input, Conv1D, MaxPooling1D,
    Flatten, Dense, Dropout
)
from tensorflow.keras.callbacks import EarlyStopping
from sklearn.utils.class_weight import compute_class_weight

tf.keras.utils.set_random_seed(42)

# A 1D CNN expects (samples, features, channels)
X_train_cnn = X_train[..., np.newaxis]
X_val_cnn = X_val[..., np.newaxis]
X_test_cnn = X_test[..., np.newaxis]

classes = np.unique(y_train)
weights = compute_class_weight(
    class_weight="balanced",
    classes=classes,
    y=y_train
)
class_weights = dict(zip(classes.tolist(), weights.tolist()))

model = Sequential([
    Input(shape=(X_train.shape[1], 1)),
    Conv1D(32, kernel_size=3, activation="relu", padding="same"),
    MaxPooling1D(pool_size=2),
    Conv1D(64, kernel_size=3, activation="relu", padding="same"),
    Flatten(),
    Dense(64, activation="relu"),
    Dropout(0.3),
    Dense(1, activation="sigmoid")
])

model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),
    loss="binary_crossentropy",
    metrics=[
        "accuracy",
        tf.keras.metrics.Precision(name="precision"),
        tf.keras.metrics.Recall(name="recall")
    ]
)

model.summary()

early_stop = EarlyStopping(
    monitor="val_loss",
    patience=2,
    restore_best_weights=True
)

history = model.fit(
    X_train_cnn,
    y_train,
    validation_data=(X_val_cnn, y_val),
    epochs=8,
    batch_size=512,
    class_weight=class_weights,
    callbacks=[early_stop],
    shuffle=False,
```

```
    verbose=1
)
```

Model: "sequential"

Layer (type)	Output Shape	Param #
conv1d (Conv1D)	(None, 78, 32)	128
max_pooling1d (MaxPooling1D)	(None, 39, 32)	0
conv1d_1 (Conv1D)	(None, 39, 64)	6,208
flatten (Flatten)	(None, 2496)	0
dense (Dense)	(None, 64)	159,808
dropout (Dropout)	(None, 64)	0
dense_1 (Dense)	(None, 1)	65

Total params: 166,209 (649.25 KB)
Trainable params: 166,209 (649.25 KB)
Non-trainable params: 0 (0.00 B)

```
Epoch 1/8
235/235 ━━━━━━━━━━━━━━━━━━━━ 17s 59ms/step - accuracy: 0.7007 - loss: 0.8191 - precision: 0.3028 - recall: 0.696
Epoch 2/8
235/235 ━━━━━━━━━━━━━━━━━━━━ 14s 58ms/step - accuracy: 0.8139 - loss: 0.5627 - precision: 0.4328 - recall: 0.600
Epoch 3/8
235/235 ━━━━━━━━━━━━━━━━━━━━ 14s 58ms/step - accuracy: 0.8472 - loss: 0.7518 - precision: 0.5126 - recall: 0.528
Epoch 4/8
235/235 ━━━━━━━━━━━━━━━━━━━━ 23s 98ms/step - accuracy: 0.9001 - loss: 0.4443 - precision: 0.6541 - recall: 0.770
Epoch 5/8
235/235 ━━━━━━━━━━━━━━━━━━━━ 16s 67ms/step - accuracy: 0.9119 - loss: 0.4139 - precision: 0.6935 - recall: 0.785
Epoch 6/8
235/235 ━━━━━━━━━━━━━━━━━━━━ 13s 56ms/step - accuracy: 0.9287 - loss: 0.3975 - precision: 0.7680 - recall: 0.782
```

```
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.metrics import (
    accuracy_score,
    f1_score,
    recall_score,
    classification_report,
    confusion_matrix
)

probabilities = model.predict(X_test_cnn, batch_size=1024).ravel()
cnn_pred = (probabilities >= 0.5).astype(int)

print("CNN test accuracy:",
      round(accuracy_score(y_test, cnn_pred), 4))

print("CNN test macro-F1:",
      round(f1_score(y_test, cnn_pred,
                     average="macro", zero_division=0), 4))

print("Attack recall:",
      round(recall_score(y_test, cnn_pred,
                         pos_label=1, zero_division=0), 4))

print("\nCNN test classification report:")
print(classification_report(
    y_test,
    cnn_pred,
    labels=[0, 1],
    target_names=["Benign", "Attack"],
    zero_division=0
))

cm = confusion_matrix(y_test, cnn_pred, labels=[0, 1])

plt.figure(figsize=(6, 5))
sns.heatmap(
    cm,
    annot=True,
```

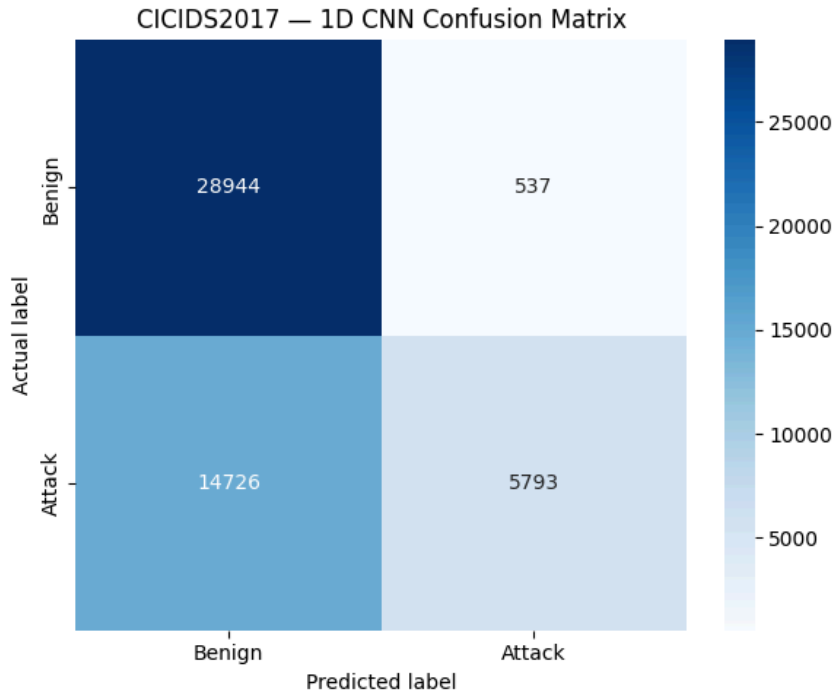
```
    fmt="d",
    cmap="Blues",
    xticklabels=["Benign", "Attack"],
    yticklabels=["Benign", "Attack"]
)
plt.xlabel("Predicted label")
plt.ylabel("Actual label")
plt.title("CICIDS2017 — 1D CNN Confusion Matrix")
plt.tight_layout()
plt.show()
```

```
49/49 ━━━━━━━━━━━━━━━━━━━━ 3s 53ms/step
CNN test accuracy: 0.6947
CNN test macro-F1: 0.6114
Attack recall: 0.2823

CNN test classification report:
              precision    recall  f1-score   support

      Benign       0.66      0.98      0.79     29481
      Attack       0.92      0.28      0.43     20519

    accuracy                           0.69     50000
   macro avg       0.79      0.63      0.61     50000
weighted avg       0.77      0.69      0.64     50000
```




```
# Training and validation loss
plt.figure(figsize=(8, 4))
plt.plot(history.history["loss"], label="Training loss")
plt.plot(history.history["val_loss"], label="Validation loss")
plt.xlabel("Epoch")
plt.ylabel("Binary cross-entropy loss")
plt.title("1D CNN Training and Validation Loss")
plt.legend()
plt.tight_layout()
plt.show()

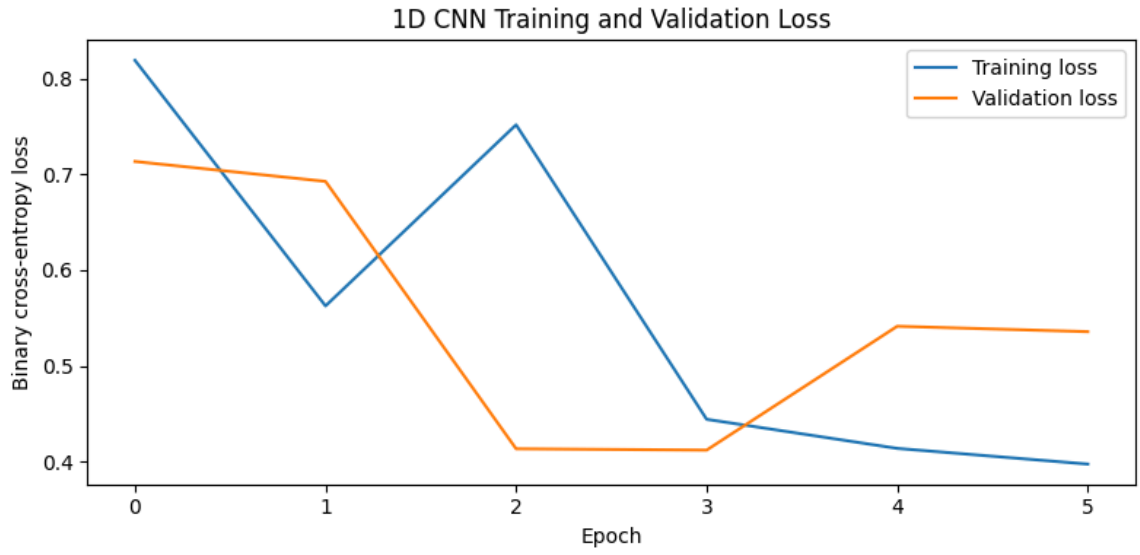
# Training and validation accuracy
plt.figure(figsize=(8, 4))
plt.plot(history.history["accuracy"], label="Training accuracy")
plt.plot(history.history["val_accuracy"], label="Validation accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.title("1D CNN Training and Validation Accuracy")
plt.legend()
plt.tight_layout()
plt.show()

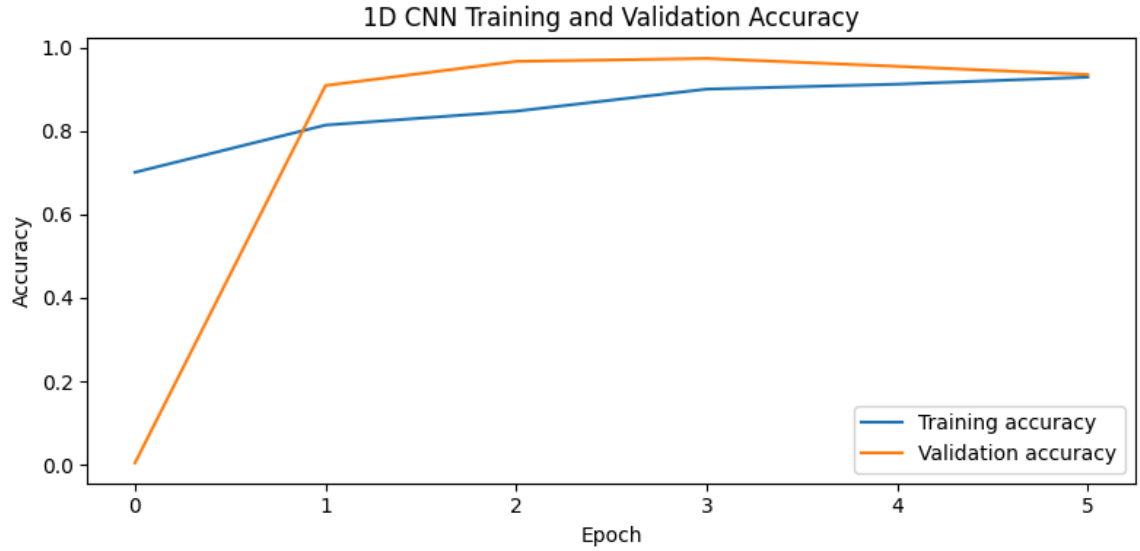
# Compare models on the same held-out test sample
```

```
baseline_test_pred = baseline.predict(X_test)

comparison = pd.DataFrame({
    "Model": ["SGD Baseline", "1D CNN"],
    "Accuracy": [
        accuracy_score(y_test, baseline_test_pred),
        accuracy_score(y_test, cnn_pred)
    ],
    "Macro-F1": [
        f1_score(y_test, baseline_test_pred,
                 average="macro", zero_division=0),
        f1_score(y_test, cnn_pred,
                 average="macro", zero_division=0)
    ],
    "Attack Recall": [
        recall_score(y_test, baseline_test_pred,
                     pos_label=1, zero_division=0),
        recall_score(y_test, cnn_pred,
                     pos_label=1, zero_division=0)
    ]
})

print(comparison.round(4).to_string(index=False))
```






```
      Model  Accuracy  Macro-F1  Attack Recall
SGD Baseline    0.7742    0.7509         0.5704
      1D CNN    0.6947    0.6114         0.2823
```

```
import joblib

model.save("cicids2017_1d_cnn.keras")

joblib.dump(
    {
```

```
        "scaler": scaler,
        "medians": medians,
        "feature_cols": feature_cols,
        "label_mapping": {
            0: "Benign",
            1: "Attack"
        }
    },
    "cicids2017_preprocessing.joblib"
)

joblib.dump(baseline, "cicids2017_baseline.joblib")

print("Saved CNN model, preprocessing objects, and baseline model.")
```

```
Saved CNN model, preprocessing objects, and baseline model.
```